Viktor Fischer*, Florent Bernard, and Nathalie Bochard

Modern random number generator design – Case study on a secured PLL-based TRNG

https://doi.org/10.1515/itit-2018-0025
Received September 28, 2018; revised December 16, 2018; accepted December 21, 2018

Abstract: Random number generators (RNGs) are basic cryptographic primitives. They are used to generate cryptographic keys, initialization vectors, challenges and nonces in cryptographic protocols, and random masks in countermeasures against side channel attacks. RNGs designed for cryptography must generate unpredictable random numbers. According to recent security standards, the unpredictability of generated random numbers must be thoroughly evaluated. In this paper, we provide a concrete example – a phase-locked loop based RNG protected by novel dedicated embedded tests, on which we show how stringent security requirements including unpredictability of generated numbers can be met, while respecting the standards.

Keywords: Random number generators, random bit generators, entropy, physical security, hardware security

ACM CCS: Security and privacy → Security in hardware → Hardware security implementation, Security and privacy → Security in hardware → Embedded systems security, Security and privacy → Cryptography → Key management

1 Introduction

Random number generators (RNGs) belong among basic cryptographic primitives. Random numbers are essential in cryptography. They are used not only as confidential keys, but also as initialization vectors, challenges, nonces, and random masks in side channel attack countermeasures. RNGs designed for cryptography must generate unpredictable random numbers – having perfect statistical properties is necessary but not sufficient.

There are two main categories of RNGs: deterministic random number generators (DRNGs) and true random number generators (TRNGs). DRNGs are based on algorithmic processes and are consequently not truly random. TRNGs exploit unpredictable processes, such as analog phenomena in electronic devices, to produce a random binary sequence or a sequence of random numbers. The unpredictability of DRNGs is guaranteed computationally while that of TRNGs is guaranteed physically. A good knowledge of the underlying physical process in TRNG that ensures its randomness and hence its unpredictability is therefore necessary. Hybrid RNGs (HRNGs) combine the advantages of the two RNG classes: the physical unpredictability of the TRNG and the statistical quality of the DRNG.

The statistical quality of TRNGs and DRNGs is usually evaluated using statistical test suites such as the one proposed by George Marsaglia [1] and extended by the American National Institute of Standards and Technology NIST in SP 800-22 [2]. The goal of these suites is to detect statistical weaknesses including non-uniformity or existence of any patterns in the generated random sequence of only limited size. In no case can these tests guarantee the unpredictability of random binary sequences.

In recent decades, to guarantee security, certification authorities have worked together with experts from academia and industry worldwide on security standards for cryptographic algorithms, such as block ciphers, digital signature algorithms, hashing functions, and authenticated encryption. However, physical random number generators rely on physical phenomena, which, in turn, depend on the underlying technology and consequently cannot be standardized. Therefore, instead of searching for a standard TRNG principle, the authorities propose a standard design and evaluation approach.

Currently, two standard approaches exist: German AIS 20/31 [3] and American NIST SP 800-90, part A, B, and C [4, 5, 6]. Since they are developed independently, they are slightly different. Moreover, they are still evolving (especially the American documents). These differences can cause difficulties in TRNG designs.

***Corresponding author: Viktor Fischer,** Hubert Curien Laboratory UMR 5516 CNRS, Jean Monnet University in Saint-Etienne, member of University of Lyon, Saint-Etienne, France; and Faculty of Information Technology, Czech Technical University in Prague, Prague, Czech Republic, e-mail: fischer@univ-st-etienne.fr, ORCID: https://orcid.org/0000-0002-2574-0924
Florent Bernard, Nathalie Bochard, Hubert Curien Laboratory UMR 5516 CNRS, Jean Monnet University in Saint-Etienne, member of University of Lyon, Saint-Etienne, France, e-mails: florent.bernard@univ-st-etienne.fr, nathalie.bochard@univ-st-etienne.fr

Brought to you by | Göteborg University - University of Gothenburg
Authenticated
Download Date | 1/25/19 1:16 PM

The objective of this paper is to illustrate how strict security requirements can be met, while aiming at compliance with both approaches. Moreover, we demonstrate that the new embedded tests dedicated to the PLL-TRNG, which are based on the stochastic model of the generator, are faster and more efficient than commonly used black-box statistical tests, since they can indicate security margins regarding required entropy bounds.

2 From a past to a modern TRNG design

The first TRNG evaluation approach was based on American Federal Information Processing Standard (FIPS) 140-1 [7], that included recommendations for the RNG design and four simple black-box statistical tests. This standard was subsequently superseded by FIPS 140-2. The new version contained a number of small updates and the four statistical tests were no longer required.

In 2001, the German Federal Office for Information Security (German acronym BSI) proposed a new standard for secure RNG design – ASI 20/31, updated in 2011 [3]. In contrast to FIPS 140-1 and 2, AIS 20/31 is dedicated solely to the RNG design. Depending on the security level targeted, it requires the designer to carefully analyze and describe the source of randomness, to estimate entropy using a stochastic model, and to propose TRNG-specific embedded tests based on its stochastic model. The standard specifies the methodology the security evaluator should apply when evaluating the generator, including procedures of offline statistical testing (Procedures A and B).

In contrast to the German approach, the American standard NIST SP 800-90 A to C [4, 5, 6] requires neither a statistical model, nor dedicated tests. Instead of using the model, entropy is estimated using black-box tests and a testing strategy.[1] The designer can propose dedicated tests, but they must be proven to be at least as efficient as the required continuous black-box tests.

It consequently seems to be quite difficult to design a generator, that would fully comply with both standards, without additional design and area costs. To help the designers, we show how to move toward compliance by using the extended security approach presented in Fig. 1. The proposed RNG complies with AIS 20/31 (i. e. dedicated tests based on the model are used to monitor the source of randomness) and includes NIST continuous tests. The NIST tests and known answer tests (KATs) verify the integrity of the whole data path of the RNG.

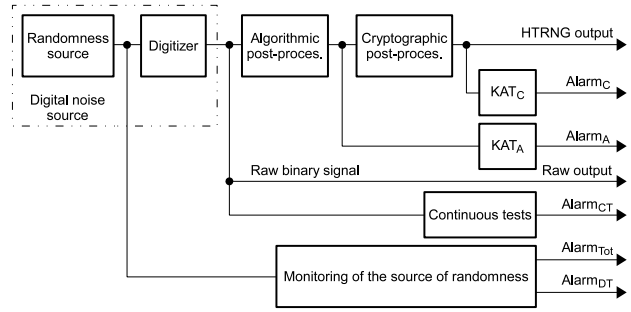

Figure 1: Extended security approach to the TRNG design including embedded tests, based on AIS 20/31 recommendations and including NIST continuous tests.

3 PLL-based TRNG

The PLL-TRNG uses phase-locked loops (PLLs) as source of randomness and a coherent sampling method to convert it into random numbers. In coherent sampling, both sampled and sampling signals are periodic signals with a known frequency ratio. In our case, both signals are binary clock signals. In the PLL-TRNG, the relationship between the frequencies of the sampling and sampled clock signals is guaranteed by the PLL.

3.1 PLL as a jittered clock generator

In logic devices, PLLs are often used as frequency synthesizers in clock generators. They generate a clock signal (a periodic rectangular waveform signal) of frequency f_{out} from the input clock signal of frequency f_{in}:

$$\frac{f_{out}}{f_{in}} = \frac{K_M}{K_D}, \tag{1}$$

where K_M and K_D are frequency multiplication and division factors of the PLL (see Fig. 2).

The phase/frequency detector (PFD) compares the phase/frequency of the input clock signal (clk_{in}) divided by N with the phase/frequency of the clock generated in the voltage-controlled oscillator (VCO) divided by M. The output of the detector charges and discharges an internal capacitor using a charge pump (CP). The capacitor voltage is then smoothed using the loop filter (LF) and the

1 Note that the latest version of the standard accepts the use of the model to verify the claimed entropy rate, but it is not explicitely required.

Brought to you by | Göteborg University - University of Gothenburg
Authenticated
Download Date | 1/25/19 1:16 PM

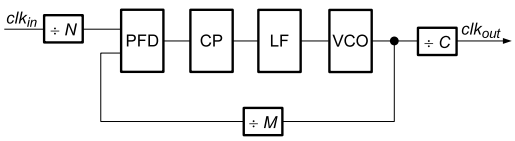

Figure 2: Block diagram of a phase-locked loop: N, M, C – input, loop, and output frequency divisors, respectively; PFD – Phase/frequency detector; CP – Charge pump; LF – Loop filter; VCO – Voltage-controlled oscillator.

smoothed voltage controls the frequency of the VCO. This frequency is thus proportional to the PLL input frequency. Since the VCO output frequency is divided by C, the output frequency of the PLL is equal to $f_{out} = f_{in} \cdot K_M/K_D = f_{in} \cdot M/(N \cdot C)$.

The main contributor to the PLL output clock jitter is the VCO, but other PLL blocks are also important. Study of the PLL as a source of randomness is an important phase of the PLL-TRNG design, but is beyond the scope of this paper. The aim of the designer should be to reduce the jitter of the PLL input clock to the minimum (for example by using a low-jitter quartz oscillator) and to setup PLL parameters to increase the impact of the thermal noise on the output clock jitter to the maximum.

3.2 Principle of the PLL-based TRNG

The TRNG exploiting the jitter introduced by the PLL, which is presented in Fig. 3, was first proposed in [8] and enhanced in [9].

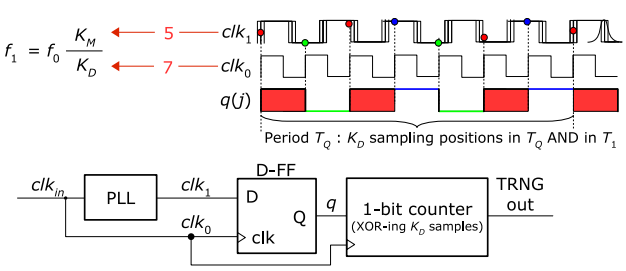

Figure 3: Principle of the PLL-based TRNG.

The jittered clock signal clk_1 from the PLL is sampled in a D flip-flop (D-FF) using the reference clock signal clk_0. The 1-bit counter counts the number of samples equal to one. Because of the frequency relationship fixed by the PLL, a pattern with a period $T_Q = K_D \cdot T_0 = K_M \cdot T_1$ appears at the output of the flip-flop. Consequently, some samples are always equal to one (blue samples in Fig. 3), some are always equal to zero (green samples) and some are random (red samples).

Based on the coherent sampling principle, by rearranging the samples according to their position, the form of one period of signal clk_1 can be reconstructed. We will see in Sec. 4.3 that this fact can be used to monitor jitter and entropy.

The PLL-TRNG output bit rate is $R = T_Q^{-1} = f_0/K_D$ and its sensitivity to the jitter is $S = \Delta^{-1} = K_D/T_1$. The sensitivity to the jitter thus increases with number of samples per period T_1 and is inversely proportional to the distance between samples in period T_1 (i. e. Δ).

In contrast to other TRNGs, the design of the PLL-TRNG has no requirements regarding placement and routing of its components. It is based only on selecting suitable PLL multiplication and division factors. Nevertheless, this choice is sometimes not straightforward, because it is physically limited by the characteristics of the PLL, such as maximum and minimum values of N, M, and C, and of input, output, PFD, and VCO frequency ranges. The choice of these parameters is analyzed and the solution is proposed in [10].

If the available ranges of physical parameters are not sufficient to obtain the required bit and/or entropy rate, two PLLs can be used as depicted in Fig. 4.

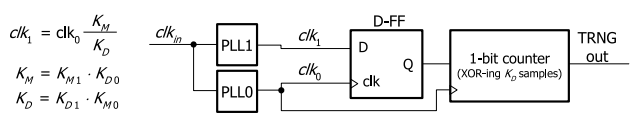

Figure 4: Parallel configuration of PLL-TRNG with two PLLs.

4 Toward a secure PLL-TRNG design

In this section, we demonstrate how the new approach to TRNG design can improve security. We use six hardware architectures, in which the RNG represents a part of the cryptographic system including the AES cipher and the USB data interface. In four of six architectures (Configuration 1 to 4), the PLL-TRNG uses two PLLs as shown in the top panel of Fig. 5. In the two remaining configurations, the PLL-TRNG uses only one PLL (Fig. 5 b) and c)). However, as can be seen in Fig. 5, in all cases, the system needs a total of two PLLs, since one PLL is always needed to generate clock for the cipher and USB interface. All the configurations were implemented on a hardware platform dedicated to TRNG evaluation with Altera Cyclone III FPGA device [11].

Brought to you by | Göteborg University - University of Gothenburg
Authenticated
Download Date | 1/25/19 1:16 PM

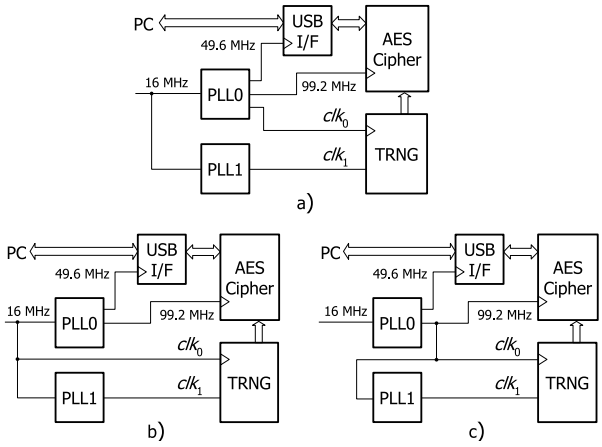

Figure 5: Architectures of the PLL-TRNG configurations tested.

In the first four configurations corresponding to the architecture in Fig. 5 a), both PLLs received the same low jitter input clock signal (16 MHz) from a quartz oscillator. PLL0 was used to generate clock signals for the USB interface (49.6 MHz), AES cipher (99.2 MHz), and the TRNG reference clock clk_0 (99.2 MHz). In the two last configurations, PLL0 was used in the same way as previously, but the relationship between clk_0 and clk_1 depended only on PLL1. Consequently the PLL-TRNG itself used only one PLL (PLL1), but with different input frequencies (16 MHz and 99.2 MHz, respectively) and with different multiplication and division factors. The PLL-TRNG parameters are detailed in Tab. 1.

Table 1: PLL-TRNG parameters in the six configurations tested.

#	K_M	K_D	clk_0 [MHz]	clk_1 [MHz]	R [kb/s]	Δ [ps]
1	1015	279	99.2	360.1	**355**	**9.9**
2	185	217	99.2	84.6	**457**	**54.5**
3	185	93	99.2	197.3	**1067**	**54.5**
4	595	93	99.2	634.7	**1067**	**16.9**
5	244	9	16	433.8	**1778**	**256.1**
6	214	205	248	258.9	**1210**	**18.8**

As can be seen in Tab. 1, not only the bit rate (R), but also the distance between samples ($\Delta = S^{-1}$) differ considerably. We can deduce that Configurations 1 and 4 probably have the highest entropy rate and Configuration 5 the lowest. But since we do not know the size of the jitter, which differs in all configurations, we cannot draw any conclusion about the output entropy rate.

In the following sections, we first analyze the parameters of the six PLL-TRNG configurations using a standard approach that consists in evaluating TRNG using black-box statistical tests, and then using the new security approach based on a stochastic model and dedicated tests based on the model.

4.1 Evaluation of the six TRNG configurations using an old style approach

Two random data outputs are available in all the configurations studied: a raw random bit stream obtained at the output of the 1-bit counter and cryptographically post-processed data using the AES cipher in the counter mode, which represents an approved CTR_DRBG from [4]. In this first step, we applied a common strategy of TRNG testing using black-box statistical tests. We used the FIPS 140-1 test suite. Of course, more precise and more time consuming test suites such NIST SP 800-22 or Diehard could be used, but at this stage of the design, we only need a rough quality evaluation of RNG. The results are presented in Tab. 2.

Table 2: Results of statistical testing of the selected PLL-TRNG configurations using the FIPS 140-1 test suite (the last two columns contain the results of the tests of the raw binary signal and of the cryptographically post-processed signal, respectively).

#	K_M	K_D	R [kb/s]	Δ [ps]	Raw s. testing	PP s. testing
1	1015	279	355	9.9	passed	passed
2	185	217	457	54.5	failed	passed
3	185	93	1067	54.5	failed	passed
4	595	93	1067	16.9	passed	passed
5	244	9	1778	256.1	failed	passed
6	214	205	1210	18.8	passed	passed

As could be expected, the cryptographically post-processed bit stream passes all the tests. However, this result does not provide any information on the entropy rate per bit (our cryptographic post-processing does not increase the entropy rate).

On the other hand, the results of testing the raw binary signal coming from Configurations 1 and 4 fulfilled the above-mentioned expectations. Moreover, the raw signal obtained from Configuration 6 passes the statistical tests. However, the security margin in individual configurations, which should guarantee good results independently of environmental conditions, was not known. Next, we will see how the new TRNG evaluation approach can help solve this problem.

Brought to you by | Göteborg University - University of Gothenburg
Authenticated
Download Date | 1/25/19 1:16 PM

4.2 Entropy estimation using the stochastic model

To estimate the entropy rate at the generator output, we use the model published in [12]. According to the model, we first sort realizations of the phase φ_i of the clock signal clk_1 (assuming that K_M and K_D are mutually prime) following the next equation:

$$j = i \cdot K_M \mod K_D \tag{2}$$

This sorting, which is based on the coherent sampling principle, serves to reconstruct the original form of the sampled signal from the acquired samples, as illustrated in Fig. 6.

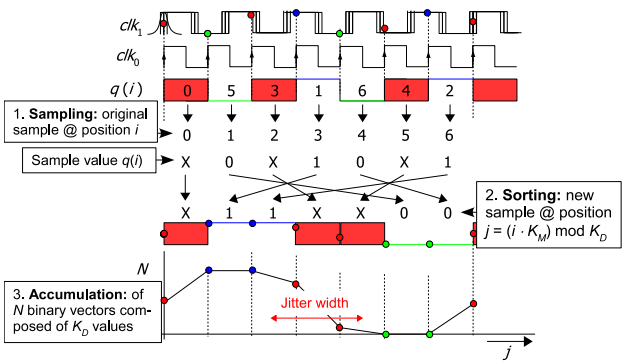

Figure 6: Illustration of the stochastic model: samples are sorted and accumulated to compute probability of a sample to be equal to one ($P(X_j = 1)$).

After sorting, the phase of the sample with index j can be viewed as a realization φ_j of the random variable ϕ_j defined as follows:

$$\phi_j \sim \mathcal{N}(\varphi_0 + j \times \Delta \mod T_1, \sigma) = \mathcal{N}(\mu_j, \sigma), \tag{3}$$

where $\Delta = T_1/K_D$ is the mean distance between realizations φ_j and φ_{j+1} of the random variable ϕ_j, μ_j is the mean value of ϕ_j and σ its standard deviation.

Although the mean value μ_j is from the interval $\mu_j \in [0, T_1)$, realizations of ϕ_j can be outside this interval because of the jitter σ. We note H_1 the half-period of clk_1. We can set the initial phase φ_0 so that the sampled signal is equal to '1' in $[0, H_1]$ and equal to '0' in (H_1, T_1). Then the probability to sample a '1' (K_D sample values in period T_Q: $X_0, \ldots, X_{K_D-1}$) is given as:

$$\begin{aligned} &\text{for } j \in \{0, \ldots, K_D - 1\}, \\ &P(X_j = 1) = P(0 < \phi_j < H_1) + P(T_1 < \phi_j < T_1 + H_1). \end{aligned} \tag{4}$$

Condition $\phi_j < T_1 + H_1$ should be always fulfilled because $\sigma << H_1$, therefore $P(\phi_j < T_1 + H_1)$ should be close to one.

For any $t \in \mathbb{R}$, $P(\phi_j < t) = \frac{1}{\sqrt{2\pi}\sigma} \int_{-\infty}^{t} e^{-\frac{(x-\mu_j)^2}{2\sigma^2}} dx$, consequently, we can express the probability that the random variables X_j will be equal to one as follows:

$$P(X_j = 1) = 1 - \frac{1}{\sqrt{2\pi}\sigma} \cdot \left(\int_{-\infty}^{T_1} e^{-\frac{(x-\mu_j)^2}{2\sigma^2}} dx - \int_{0}^{H_1} e^{-\frac{(x-\mu_j)^2}{2\sigma^2}} dx \right). \tag{5}$$

Using Eq. (5), we can reconstruct the cumulative distribution function (CDF) for the whole period T_Q. Figure 7 depicts this function constructed for $\sigma = 9.60$ ps, $\varphi_0 = T_1/4$, $K_D = 93$, $K_M = 595$ (corresponding to Configuration 4).

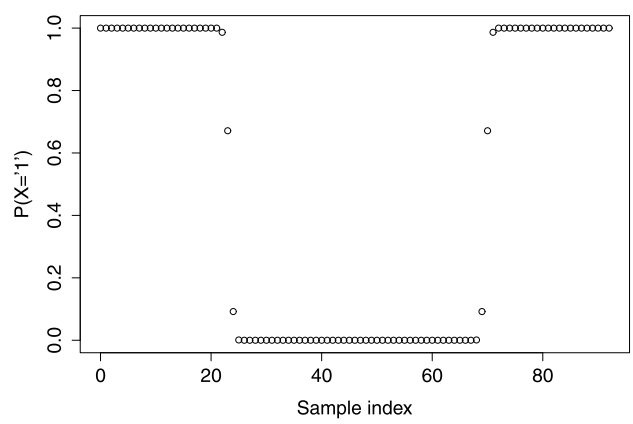

Figure 7: Cumulative distribution function of the period T_Q based on the stochastic model from [12] for $\sigma = 9.60$ ps, $\varphi_0 = T_1/4$, $K_D = 93$, $K_M = 595$.

It will be recalled that the PLL-TRNG output is obtained by counting samples equal to one, i. e. by XOR-ing K_D samples during one period T_Q. The PLL-TRNG output is thus random variable $Y = X_0 \oplus X_1 \oplus \ldots X_{K_D-1}$. According to the analysis made by Davies in [13] and assuming independence of X_i, the probability that the TRNG output will be equal to one is determined by the following equation:

$$P(Y = 1) = \frac{1}{2} + B, \tag{6}$$

where

$$B = (-2)^{K_D-1} \prod_{k=0}^{K_D-1} \left(P(X_k = 1) - \frac{1}{2} \right) \tag{7}$$

represents the bias of the generator output, ideally equal to zero. From Eq. (7) it is clear that zero bias is obtained if

Brought to you by | Göteborg University - University of Gothenburg
Authenticated
Download Date | 1/25/19 1:16 PM

any of the samples X_j have the probability $P(X_j = 1)$ equal to 1/2. From Eq. (6), we can compute the Shannon entropy rate $H(Y)$ of the TRNG output:

$$H(Y) = -P(Y = 1)\log_2(P(Y = 1)) - (1 - P(Y = 1))\log_2(1 - P(Y = 1)) \quad (8)$$

Using Eq. (2), (3), and (5) and knowing f_0, K_M and K_D, we can compute $P(Y = 1)$ and $H(Y)$ as a function of the jitter size (σ) for all six versions of the TRNG studied. The plots of the output entropy rates of the six generator versions are shown in Fig. 8.

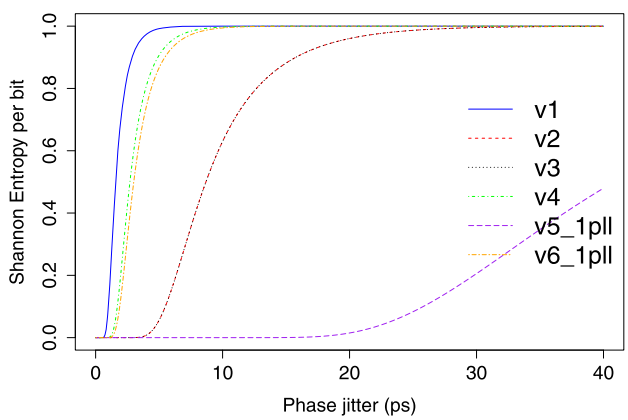

Figure 8: Plots of output entropy rates of the six versions of PLL-TRNG.

Our stochastic model specifies the entropy rate per output bit depending on the jitter size (σ) as the main input parameter. The model confirms that Configurations 1, 4 and 6 need a much smaller jitter to reach an entropy rate close to 1 (the ideal value). On the contrary, Configuration 5 would need unreal jitter values to reach the required entropy rate. To estimate the entropy rate per TRNG output bit, we thus need to measure the size of the jitter (σ).

4.3 Jitter measurement for entropy rate estimation

The tracking jitter of the PLL, which is used as the source of randomness, can be quantified using the principle of the coherent sampling presented in Fig. 6. This figure shows clearly that the size of the jitter is proportional to the width of the rising and falling edge.

Using this fact, we measured the jitter by sending the sampled jittered clock data (subsequent samples $q(j)$ of the clock signal clk_1) via the USB interface to the PC. Plots of the CDFs for Configurations 3 and 4 obtained by sorting the samples using Eq. (2) and accumulating $N = 255$ periods T_Q of K_D samples are shown in Fig. 9.

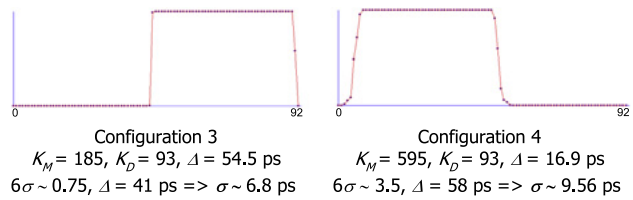

Figure 9: Results of external jitter measurement in Configuration 3 and 4.

As it was explained in Section 4.2, phase ϕ_j follows normal distribution $\mathcal{N}(\mu_j, \sigma)$ and each edge of the signal (rising or falling) can be described by a CDF.

Since 99.74 % of the population, which follows the normal distribution, is in the range of 6σ, the width of the rising or falling edge (expressed as a function of number of unstable samples n and the distance between samples Δ) is equal approximately to 6σ, i. e. $6\sigma \sim n\Delta$.

Figure 9 shows that $6\sigma \sim 0.75\Delta$ in Configuration 3 and $6\sigma \sim 3.5\Delta$ in Configuration 4. This corresponds to $\sigma \sim 6.80$ ps and $\sigma \sim 9.56$ ps, respectively. Using the same principle, we quantified the jitter in all six PLL-TRNG configurations. The results are presented in Tab. 3.

Table 3: Results of the jitter measurement using the proposed method in six TRNG configurations (last column).

#	K_M	K_D	R [kb/s]	Δ [ps]	σ [ps]
1	1015	279	355	9.9	8.72
2	185	217	457	54.5	10.20
3	185	93	1067	54.5	6.80
4	595	93	1067	16.9	9.56
5	244	9	1778	256.1	0.00
6	214	205	1210	18.8	39.9

Next, knowing the size of the tracking jitter, we were able to estimate the entropy rate using the stochastic model. The results are depicted graphically in Fig. 10.

We can see that for $\sigma \sim 8.72$ ps and $\sigma \sim 39.9$ ps, the Shannon entropy rate at the output of Configuration 1 and 6 is equal to 1 with a significant security margin. On the other hand, the security margin in Configuration 4 with $\sigma \sim 9.56$ ps is by far smaller. Moreover, Configurations 2 and 3 feature some intermediate entropy rates, that would

Brought to you by | Göteborg University - University of Gothenburg
Authenticated
Download Date | 1/25/19 1:16 PM

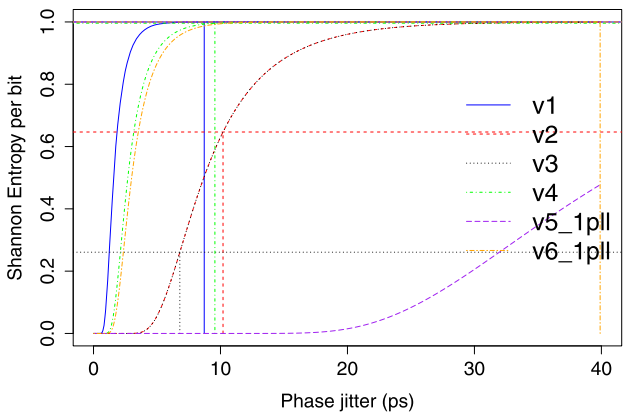

Figure 10: Entropy rate estimation for the six PLL-TRNG configurations. Vertical lines represent jitter sizes from Tab. 3.

be difficult to estimate using black-box statistical tests. It would therefore be difficult for the designer to estimate the effort needed to reach the required entropy rate.

Table 4: Entropy rate estimation using the stochastic model and the results of statistical testing using Procedure A and Procedure B of the AIS 20/31 test suite.

#	Δ [ps]	σ [ps]	H(Y)	AIS20/31 Proc. A	AIS20/31 Proc. B
1	9.9	8.72	0.998	passed	passed
2	54.5	10.20	0.650	failed	failed
3	54.5	6.80	0.260	failed	failed
4	16.9	9.56	0.997	passed	passed
5	256.1	–	0.000	failed	failed
6	18.8	39.9	0.998	passed	passed

The results of jitter measurements, entropy estimations and AIS 20/31 Procedure A and B test results are presented in Tab. 4. It can be seen that the entropy rate estimated using the model corresponds perfectly to the results of the AIS 20/31 test suite. Since the algorithm of jitter measurement is relatively simple, it can be implemented inside the device to monitor the jitter and hence the entropy rate in real time. The embedded entropy evaluation method is presented in the following section.

4.4 From jitter measurement to dedicated tests based on the stochastic model

To test the quality of generated numbers, we suggest measuring two parameters, which depend on the size of the jitter: parameters P_1 and P_2. Comparisons of values of these two parameters with predefined thresholds will then constitute stochastic tests dedicated to the PLL-TRNG.

Figure 11 illustrates definition of parameter P_1, which corresponds to the number of samples with probability $\mathrm{P}(X_j = 1) \in (\mathrm{P}_{min}; \mathrm{P}_{max})$, obtained within the period T_Q:

$$P_1 = \#\{(X_j)_j | \mathrm{P}(X_j = 1) \in (\mathrm{P}_{min}; \mathrm{P}_{max})\}. \tag{9}$$

Probability bounds P_{min} and P_{max} are selected depending on number N of periods T_Q, which are accumulated to compute $\mathrm{P}(X_j = 1)$.

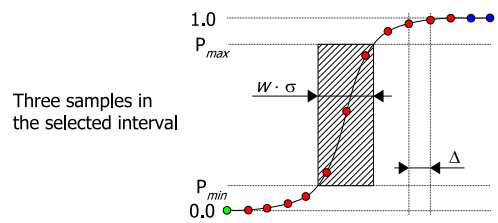

Figure 11: Definition of parameter P_1: number of random samples with probability $\mathrm{P}(X_j = 1) \in (\mathrm{P}_{min}; \mathrm{P}_{max})$, i. e. three samples in this figure.

Parameter P_2 is defined as follows:

$$P_2 = \sum_{j=0}^{K_D-1} P(X_j = 1)(1 - P(X_j = 1)). \tag{10}$$

For the given number of periods T_Q taken into account (N), the value of parameter P_2 can be estimated using the following equation:

$$P_2 = \frac{1}{N^2} \sum_{j=0}^{K_D-1} A_j(N - A_j), \tag{11}$$

where A_j represents number of cases during N periods T_Q, in which the random sample X_j was equal to one.

Unlike parameter P_1, which depends only on the number of unstable samples, parameter P_2 depends also on their position – it reaches its maximal value if the probability of one of samples is equal to 0.5.

Again, we acquired the sampled jittered clock data (the output of the D flip-flop) to the host PC and then used the jitter quantification method based on the sorting and accumulation of samples presented in Section 4.3. We then computed values of parameters P_1 and P_2 according to Eq. (9) and (10), respectively. The obtained values for the six TRNG configurations are listed in Tab. 5.

The last task to complete in designing the dedicated tests was to specify the value of parameters P_1 and P_2,

Brought to you by | Göteborg University - University of Gothenburg
Authenticated
Download Date | 1/25/19 1:16 PM

Table 5: Entropy estimation using the model and values of parameters P_1 and P_2 compared to results of statistical testing using Procedure B of AIS 20/31.

#	H(*Y*)	P_1	P_2 [$\cdot 10^3$]	AIS20/31 Proc. B
1	0.998	6–8	53–56	passed
2	0.650	1–2	8–9	failed
3	0.260	1	10–11	failed
4	0.997	4–5	38–42	passed
5	0.000	0	1–2	failed
6	0.998	15–18	120–140	passed

which would guarantee the required entropy rate. Once again, the stochastic model helped us.

Using the model and knowing f_0, K_M, and K_D, we defined the minimum required jitter size and hence the border values of P_1 and P_2 for all TRNG configurations. The general principle is the following:

1. The Shannon entropy per bit is a monotonic function of σ. Therefore, a minimum value $\sigma_{0.997}$ of σ that guarantees the required entropy bound (i. e. 0.997 according to AIS 20/31) can be determined using Eq. (5), (6), and (8).
2. This minimum jitter value is used to compute the minimum number of unstable samples according to Eq. (9) (threshold of parameter P_1).
3. The same jitter value is used to compute the lower bound of parameter P_2 using Eq. (10).

Interestingly, we observed that the same thresholds Tr_{P_1} and Tr_{P_2} guarantee sufficient entropy rate in all PLL-TRNG configurations:

$$P_1 > Tr_{P_1} = 4, \tag{12}$$

$$P_2 > Tr_{P_2} = 35,500. \tag{13}$$

Indeed, Tab. 5 shows that outputs of TRNG configurations, in which parameters P_1 and P_2 satisfy both conditions, also pass the Procedure B test suite. This result confirms suitability of the use of parameters P_1 and P_2 as a basis for dedicated embedded tests, which are much faster than Procedure B of AIS 31. Based on the presented results, we selected Configuration 1 as the best candidate for the PLL-TRNG core, since it featured the biggest security margin.

4.5 Setup and implementation of dedicated embedded tests

To maintain compliance with AIS 20/31, the designer is required to propose two kinds of tests:

- *Total failure test*, which should be fast enough while featuring small probability of false alarms;
- *Dedicated test(s)*, which should be able to detect (with a sufficient precision) intolerable weaknesses of the generator.

We propose to use two dedicated tests:

- Test T_1 based on parameter P_1 will constitute a heart of the Total failure test,
- Test T_2 based on parameter P_2 will be used as a dedicated online test.

Test T_1 will count number of random samples during $N = 255$ periods T_Q having probability $\mathrm{P}(X_j = 1) \in (0.125; 0.875)$ according to Eq. (9). The probability bounds of the samples were selected to avoid truncation, with respect to the fact that the accumulation of samples is controlled with an 8-bit counter and consequently, we let $\mathrm{P}_{min} = 32/256$ and $\mathrm{P}_{max} = (256 - 32)/256$.

The Total failure alarm is triggered:

- If one of the two PLLs is not locked,
- If Test T_1 fails, i. e. no random sample was detected during $N = 255$ periods T_Q and consequently $P_1 = 0$.

Test T_2 dedicated to the PLL-TRNG compares the value of parameter P_2 with the threshold Tr_{P_2} according to Eq. (13). To reduce the cost of the test, the value of P_2 and the threshold is divided by 256, so the modified threshold is $Tr'_{P_2} = Tr_{P_2}/256 = 139$.

The architecture of the secured PLL-TRNG core including the Total failure test and dedicated test T_2 is presented in Fig. 12. To compute values of P_1 and P_2, K_D binary samples are accumulated in embedded memory during 255 periods T_Q. To get the value of P_1, the accumulated sample values are compared with thresholds $\mathrm{P}_{min} = 0.125$ and $\mathrm{P}_{max} = 0.875$ presented in Fig. 11. Each time the memory value is within the specified interval, the value of a 10-bit accumulator/adder, which follows the comparator, is incremented. Once the accumulation memory is traversed, the 10-bit accumulator/adder value corresponds to the value of parameter P_1.

Parameter P_2 is computed in a similar way from the contents of the accumulation memory: multiplication $A_j(N - A_j)$ is implemented using a memory as a look-up table, which gives the result of multiplication of value A_j and its one's complement. This result is accumulated in a 20-bit accumulator/adder to give the value of parameter P_2 (note that, as explained before, only 12 most significant bits are compared with the threshold Tr'_{P_2}).

The security guard continuously computes the values of parameters P_1 and P_2 and compares them with thresh-

Brought to you by | Göteborg University - University of Gothenburg
Authenticated
Download Date | 1/25/19 1:16 PM

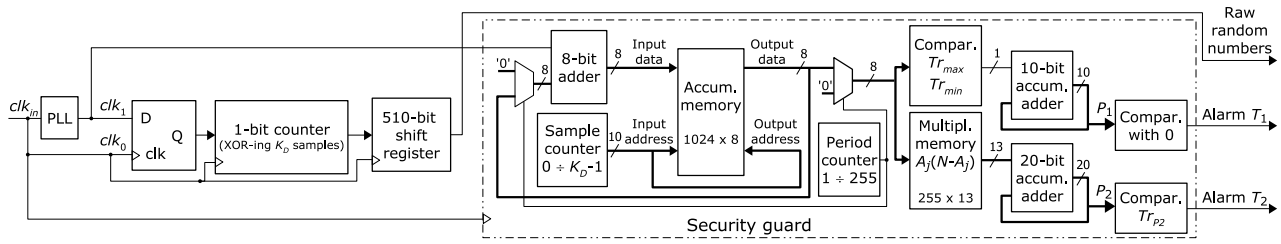

Figure 12: Block diagram of the PLL-TRNG including dedicated embedded tests.

olds ($Tr'_{P_1} = 0$ for the Total failure test and $Tr'_{P_2} = 139$ for the Dedicated online test).

In Altera Cyclone III FPGA, the security guard occupies 124 logic cells and two 9k RAM blocks. Compared to the area of the TRNG itself, the tests occupy a bigger area, but the occupied area is negligible considering the increased security provided by the security guard.

The latency of the T_1 test is 255 T_Q periods. Since the AIS 20/31 recommendations require that all the generated bits are tested by the total failure test, we added a 510-bit shift register (= 2×255 bits) before the TRNG output as presented in Fig. 12.

We should stress that using 255 periods T_Q is sufficient to evaluate parameter P_1 in test T_1, but certainly not enough for test T_2, in which a greater precision is required. To obtain the required precision, we compute the mean value of 16 successive values of parameter P_2.

4.6 Verification of efficiency of the dedicated tests

It is difficult to verify efficiency of the dedicated tests in hardware. Therefore, we did it using simulations in VHDL. Our aim was to verify that reduction of the jitter, which could be caused by aging, changing operating condition or even attacks can be detected by the embedded tests.

Figure 13 depicts output waveforms of the generator during normal operation. This normal situation is simulated by injecting the Gaussian jitter of the suitable size to both generators. Signal *strp_done* goes to one, if the startup tests (the Total failure test and Online test T_2) succeeded. We can observe that although the raw random signal is available immediately after the power-up of the generator, random data are available at the output of the shift register (signals *data_out* and *data_valid* in Fig. 13) only once the startup tests succeeded. Parameter P_1 is updated at the end of each measurement interval (255 periods T_Q).

Figure 14 depicts generator output waveforms under a weak attack. The attack starts, when the *attack* signal

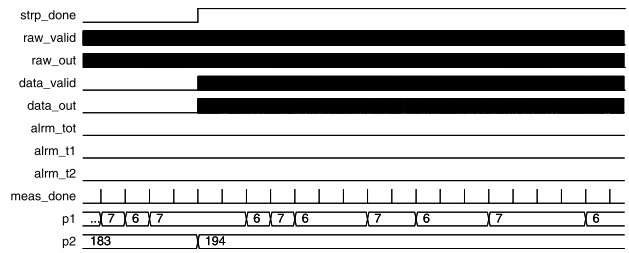

Figure 13: Behavior of the TRNG during normal operation – output is enabled once the startup tests succeeded, no alarm is triggered.

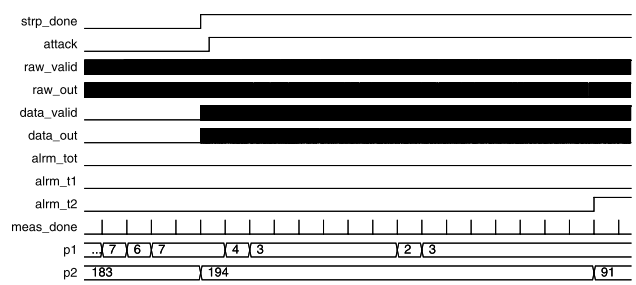

Figure 14: Verification of the *T2* test – alarm *alrm_t2* is triggered about $16 \cdot 255 = 4080$ periods T_Q after an attack or fault, but data output remains active.

goes to high (shortly after the startup testing was finished). During the attack, we reduced the injected jitter to a half of its normal size. As expected, the value of parameter P_1 decreased very quickly (its latency is 255 periods T_Q), but the parameter P_2 needed 16 times longer time to detect the attack with higher precision.

Figure 15 depicts generator output waveforms under a total failure attack. Like before, the attack starts, when the *attack* signal goes to high (i. e. soon after the end of the startup tests). During this second attack, we reduced the jitter to zero to simulate a total failure of the source of randomness. It can be seen that the generator output is stopped immediately after the attack was detected. Despite the latency of the Total failure test, thanks to the shift register placed before the output of the raw signal, no corrupted data are available at generator output.

Brought to you by | Göteborg University - University of Gothenburg
Authenticated
Download Date | 1/25/19 1:16 PM

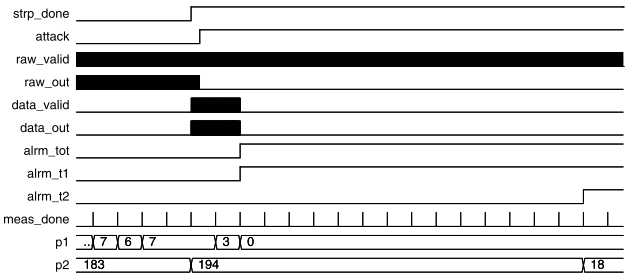

Figure 15: Verification of the *Total failure* and *T1* tests – alarms *alrm_tot* and *alrm_t1* are triggered about 512 periods T_Q after an attack or fault and data output is stopped – no erroneous data are output.

We conclude that the proposed Total failure test is fast and efficient enough to detect a catastrophic failure of the generator core. On the other hand, the Online test T_2 is precise and still sufficiently fast to detect intolerable weakness of the generator (insufficient jitter) much earlier than standard black-box statistical tests.

4.7 Security of the entire generator

Regarding the generator structure depicted in Fig. 1, we notice that because of the selection of suitable multiplication and division factors in selected Configuration 1, the algorithmic post-processing and corresponding known answer test (KAT_A in Fig. 1) is not needed.

To test the whole data path between the D-FF and cryptographic post-processing block (CPP), the input of the CPP is tested using low cost continuous statistical tests, i. e. the Repetition count test and the Adaptive proportion test required by NIST SP 800-90B. In this way, our design moved toward compliance with both European and American standards. Moreover, to guarantee security of the whole RNG, the cryptographic post-processing block is tested during the RNG initialization by a known answer test (KAT_C in Fig. 1).

Nevertheless, we wish to stress that many security-related questions concerning strategy of startup testing (order of tests, number of restarts if some test failed, etc.) and continuous monitoring and auditing of the generator remain open, since because of the limited space, they were out of the scope of our paper.

5 Conclusion

In this paper, we presented a modern TRNG design approach based on a thorough security evaluation using a stochastic model and dedicated embedded tests. By combining the secured PLL-TRNG core with simple continuous tests to test the complete TRNG data path, and with an approved cryptographic post-processing block, we obtained a modern hybrid TRNG design suitable for high end security applications. Although the importance of the novel dedicated tests is unquestionable, we believe that the presented approach, which illustrates the modern RNG design strategy, is of a comparable significance.

References

1. G. Marsaglia. *DIEHARD: Battery of Tests of Randomness*, 1996. Online. Available at: http://stat.fsu.edu/pub/diehard.
2. A. Rukhin, J. Soto, J. Nechvatal, M. Smid, E. Barker, S. Leigh, M. Levenson, M. Vangel, D. Banks, A. Heckert, J. Dray, and S. Vo. *A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications – NIST SP 800-22, rev. 1a*, 2010. Online. Available at: https://csrc.nist.gov/publications/detail/sp/800-22/rev-1a/final.
3. W. Killmann and W. Schindler. *A proposal for: Functionality classes for random number generators*, 2011. Online. Available at: https://www.bsi.bund.de.
4. E. Barker and J. Kelsey. *Recommendation for Random Number Generation Using Deterministic Random Bit Generators – NIST SP 800-90A, rev. 1a*, June 2015. Online. Available at: https://csrc.nist.gov.
5. M. S. Turan, E. Barker, J. Kelsey, K. A. McKay, M. L. Baish, and M. Boyle. *Recommendation for the Entropy Sources Used for Random Bit Generation – NIST SP 800-90B*, Jan. 2018. Online. Available at: https://csrc.nist.gov.
6. E. Barker and J. Kelsey. *Recommendation for Random Bit Generator (RBG) Constructions – NIST SP 800-90C*, April 2015. Online. Available at: https://csrc.nist.gov.
7. NIST. Federal Information Processing Standard (FIPS) 140-1 *Security Requirements for Cryptographic Modules*, January 1994. Online. Available at: https://csrc.nist.gov.
8. V. Fischer and M. Drutarovsky *True Random Number Generator Embedded in Reconfigurable Hardware*. Proceedings of the International Workshop on Cryptographic Hardware and Embedded Systems (CHES 2002), Redwood Shores, CA, USA, August 2002, pp. 415–430.
9. V. Fischer, M. Drutarovsky, M. Simka, and N. Bochard *High Performance True Random Number Generator in Altera Stratix FPLDs*. Field-Programmable Logic and Applications (FPL 2004), Leuven, Belgium, September 2004, pp. 555–564. ISBN 978-3-540-30117-2.
10. E. N. Allini, O. Petura, V. Fischer, and F. Bernard *Optimization of the PLL configuration in a PLL-based TRNG design*. Design, Automation & Test in Europe Conference & Exhibition (DATE 2018), Dresden, Germany, March 2018, pp. 1265–1270. doi: 10.23919/DATE.2018.8342209.
11. V. Fischer, P. Haddad, and F. Bernard *An open-source multi-FPGA modular system for fair benchmarking of true random number generators*. Field-Programmable Logic and

Brought to you by | Göteborg University - University of Gothenburg
Authenticated
Download Date | 1/25/19 1:16 PM

Applications (FPL 2013), Porto, Portugal, September 2013, pp. 1–4. doi: 10.1109/FPL.2013.6645570.
12. F. Bernard, V. Fischer, and B. Valtchanov *Mathematical model of physical RNGs based on coherent sampling*. Tatra Mountains Mathematical Publications Num. 1 (Vol. 45), Bratislava, Slovakia, 2010, pp. 1–14.
13. R. B. Davies *Exclusive OR (XOR) and hardware random number generators*, 2002. Online. Available at: www.robertnz.net/pdf/xor2.pdf.

Bionotes

Viktor Fischer
Hubert Curien Laboratory UMR 5516 CNRS, Jean Monnet University in Saint-Etienne, member of University of Lyon, Saint-Etienne, France
Faculty of Information Technology, Czech Technical University in Prague, Prague, Czech Republic
fischer@univ-st-etienne.fr; viktor.fischer@fit.cvut.cz

Prof. Viktor Fischer received his M. S. and Ph. D. degrees in Electrical Engineering from Technical University of Kosice in Slovakia. From 1981 to 1991 he held an Assistant Professor position at the Department of Electronics of the Technical University of Kosice. From 1991 to 2006 he was a part-time invited professor at the University of Saint-Etienne, France. From 1999 to 2006 he was also a consultant with Micronic Slovakia, oriented in hardware data security systems. From 2006 he is a full-time Professor at the University of Saint-Etienne and from November 2018 he is a part-time invited Professor at the Czech Technical University in Prague. His research interests include cryptographic engineering, secure embedded systems, and especially true random number generators embedded in logic devices.

Florent Bernard
Hubert Curien Laboratory UMR 5516 CNRS, Jean Monnet University in Saint-Etienne, member of University of Lyon, Saint-Etienne, France
florent.bernard@univ-st-etienne.fr

Dr. Florent Bernard received a B. S. degree in mathematics from the University of Angers and an M. S. degree in applied cryptography from the University of Grenoble. Since obtaining his Ph. D. degree in Computer Science from the University Paris8 in 2007, he has been working as an assistant professor at the University of Saint-Etienne. His research interests include implementation of cryptographic functions, secure embedded systems and especially modeling and evaluating true random number generators embedded in logic devices.

Nathalie Bochard
Hubert Curien Laboratory UMR 5516 CNRS, Jean Monnet University in Saint-Etienne, member of University of Lyon, Saint-Etienne, France
nathalie.bochard@univ-st-etienne.fr

Ing. Nathalie Bochard received the master's degree in electronic engineering in 1996 and the Diploma of Technological Research (DRT) in vision, telecommunication and instrumentation from the University of Lyon, in 1997. She is a research engineer at the CNRS (National Center for Scientific Research) in France, which she joined in 1998. Currently, her main research interests include embedded hardware cryptographic architectures for configurable logic devices and especially design, implementation and evaluation of true random number generators and physical unclonable functions aimed at cryptographic applications.

Brought to you by | Göteborg University - University of Gothenburg
Authenticated
Download Date | 1/25/19 1:16 PM